Digital Clock

IIT Hyderabad

Department of Electrical Engineering [ICDT]

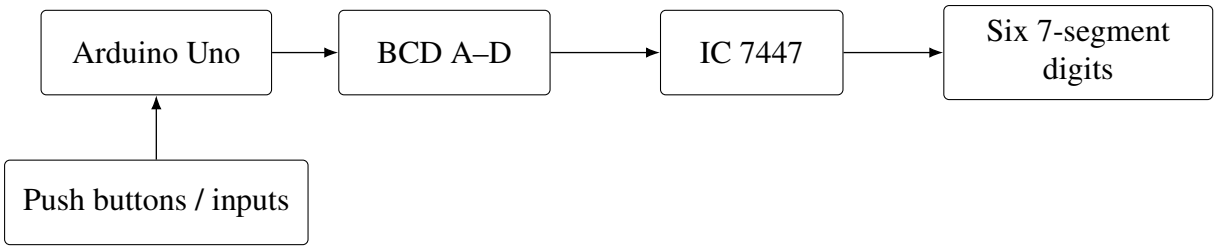
EE1010

Maths For Electrical Engineering

Prof. G. V. V. Sharma

Y. Viswa Srihas
(EE26BTECH11234)

SK. M. M. Suhan
(EE26BTECH11238)



Project focus: state machines, BCD logic, K-maps, multiplexing, switch debouncing, embedded C/C++, circuit construction, debugging, and possible phone-based Arduino interaction.

Prepared as a technical project report

Contents

1	Abstract	3
2	System Overview	3
3	Hardware Components	4
3.1	Arduino Uno	5
3.2	IC 7447	5
3.3	Seven-Segment Display	5
3.4	Push Buttons	5
3.5	Breadboard	6
3.6	Jumper Wires	6
3.7	Resistors	6
4	Pin Connections	7
4.1	Reference-project pin mapping	8
5	BCD Representation	8
6	Clock Counter and State-Machine Concept	9
7	K-Map and Boolean-Logic Understanding	7·9
8	Multiplexing	10
9	Switch Debouncing	10
10	Software Design	11
10.1	Why millis() is used	11
11	Circuit Construction Procedure	11
12	Errors and Corrections	13
13	Safety and Reliability Checks	17
14	Conclusion	18
15	References	18
A	Quick Pin-and-Logic Summary	18

1 Abstract

This project presents the design and implementation of a six-digit digital clock using an Arduino Uno, an IC 7447 BCD-to-seven-segment decoder/driver, six seven-segment display digits, push buttons, resistors, jumper wires, and a breadboard. The display represents time in the 24-hour format HH:MM:SS. The Arduino generates four shared BCD signals and selects the display digits one at a time using multiplexing. The clock counter is implemented in software using seconds, minutes, and hours with the constraints 0–59, 0–59, and 0–23 respectively.

The project is also used to demonstrate digital-design concepts rather than only programming. These concepts include state-machine thinking, BCD representation, Boolean logic, K-map based simplification, multiplexing, switch debouncing, timing, cascading counters, and practical circuit debugging. In addition, display brightness is controlled using single-tap, double-tap, and triple-tap button actions. The project photographs show the assembled prototype and the component set used for construction.

2 System Overview

The clock is divided into three major functional blocks: time generation, BCD encoding, and display driving.

$$\text{Time Counter} \rightarrow \text{BCD Encoder} \rightarrow \text{IC 7447}$$

$$\text{HH:MM:SS} \qquad A, B, C, D$$

$$\rightarrow \text{Seven-Segment Display Digits}$$

The six digits are logically arranged as:

$$\text{HH} : \text{MM} : \text{SS}$$

where the first two digits represent hours, the next two represent minutes, and the final two represent seconds. The reference project description also uses six multiplexed displays and the 23:59:59 time format.

3 Hardware Components

Component	Qty.	Purpose
Arduino Uno	1	Main microcontroller; generates timing, BCD data and digit-selection signals.
IC 7447	1	BCD-to-seven-segment decoder/driver.
Six-digit seven-segment display	1	Visual display of HH:MM:SS.
Push buttons	5	User inputs for edit/pause, digit selection, increment/decrement functions and brightness-control actions.
220 ? resistors	7	Current limiting/display protection as required by the actual wiring.
10 k? resistors	4	Pull-down resistors for push-button inputs.
Breadboard	1	Temporary circuit construction and testing.
Jumper wires	Several	Electrical connections between Arduino, breadboard, IC and displays.
USB cable	1	Power/programming connection for Arduino.

3.1 Arduino Uno

The Arduino Uno acts as the controller. It executes the embedded C/C++ program, measures elapsed time using millis(), generates the BCD bits, selects the active display digit and can read push-button inputs.

3.2 IC 7447

The IC 7447 is used as the BCD-to-seven-segment interface. Four BCD inputs, A, B, C and D, represent a decimal digit in binary-coded decimal. The decoder converts this information into the segment-driving outputs required by the seven-segment display interface.

3.3 Seven-Segment Display

Each digit contains segments conventionally labelled a through g. A decimal number is formed by turning an appropriate combination of these segments ON. Because the project has six digits, directly controlling every segment independently would require many Arduino pins. Multiplexing solves this problem.

3.4 Push Buttons

Push buttons provide user input. In the intended extended design, the buttons can be assigned to edit-mode toggle, next-digit selection, increment, decrement, and brightness control. Mechanical switches do not change state cleanly once; they may produce several rapid transitions called bounce. Therefore debouncing is necessary before treating a press as one valid event.

3.5 Breadboard

The breadboard is used as the temporary platform for constructing and testing the digital clock circuit. It provides interconnected rows and power rails so that the Arduino, 7447 IC, displays, resistors, and push buttons can be wired without permanent soldering. The breadboard also makes it easier to identify and correct wiring errors during testing.

3.6 Jumper Wires

Jumper wires are used to make electrical connections between the Arduino Uno, breadboard, 7447 IC, seven-segment displays, resistors, power rails, and push buttons. Different wire connections are used for the BCD inputs, display-enable lines, segment connections, power, and ground.

3.7 Resistors

Resistors are used to control current and protect the display and circuit components. The 220 ? resistors are used as current-limiting resistors for the display network, while the 10 k? resistors are used as pull-down resistors for the push-button inputs. The actual resistor values should be checked against the display and driver configuration before powering the circuit.

Hardware verification note

Important hardware verification note: the supplied reference document lists the seven-segment display as ?common cathode?, while the 7447 family is conventionally associated with active-low outputs and common-anode displays. This apparent inconsistency should be checked against the exact display and IC datasheet used in the physical prototype. It is a useful viva point: students should be able to explain why the electrical polarity of the display matters.

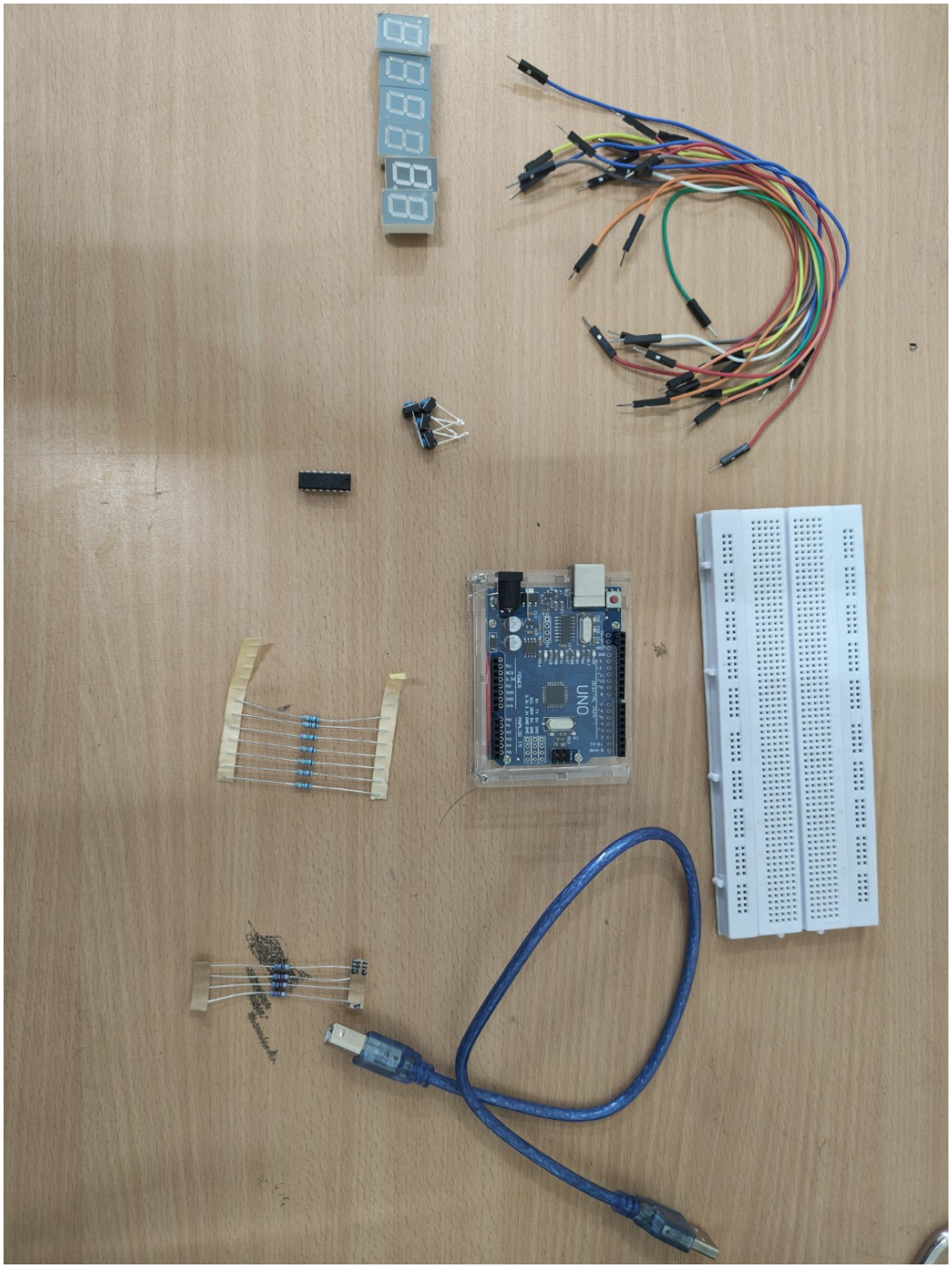


Figure 1: Hardware components used for the digital clock construction.

4 Pin Connections

The revised Arduino pin mapping is:

Arduino pin	Signal	Function
D7	A	BCD least-significant bit
D4	B	BCD bit 1
D5	C	BCD bit 2
D6	D	BCD most-significant bit
D8	Display Enable 1	Multiplexed digit selection
D9	Display Enable 2	Multiplexed digit selection
D10	Display Enable 3	Multiplexed digit selection
D11	Display Enable 4	Multiplexed digit selection
D12	Display Enable 5	Multiplexed digit selection
D13	Display Enable 6	Multiplexed digit selection

4.1 Reference-project pin mapping

The revised project uses Arduino D7, D4, D5 and D6 for the four BCD inputs of the 7447, corresponding respectively to A, B, C and D. The six display-enable lines are Arduino D8–D13. The display order is hours, minutes and seconds, with digit ranges of 0–2/0–3/0–9 for the hour positions and 0–5/0–9 for the minute and second positions.

5 BCD Representation

Binary-Coded Decimal represents each decimal digit separately using four bits. The bit weights are:

$$D = 8, \qquad C = 4, \qquad B = 2, \qquad A = 1.$$

For example,

$$5_{10} = 0101_2$$

so the BCD outputs are

$$D = 0, \qquad C = 1, \qquad B = 0, \qquad A = 1.$$

Similarly,

$$9_{10} = 1001_2.$$

Only the BCD codes 0000 through 1001 are valid decimal digits. The codes 1010 through 1111 are invalid for ordinary decimal display operation and can be treated as don't-care conditions when simplifying Boolean logic for a decimal counter.

The submitted function `writeBCD()` implements exactly this bit extraction using bitwise AND operations.

7. K-Map and Boolean-Logic Understanding

Karnaugh maps are used to simplify the Boolean logic that decides the **next state** of each clock digit. The clock does not use the same counter for every position, because seconds and minutes have a 0–59 range while hours have a 0–23 range.

7.1 What a K-map does

A K-map is a visual form of Boolean simplification. The **present-state BCD bits** are the inputs and the **next-state BCD bits** are the outputs. Instead of writing a large Boolean expression directly, the required 1s and 0s are placed in the map and neighbouring 1s are grouped. The grouped cells give a shorter Boolean expression.

X■X■ \ X■X■	00	01	11	10
00	0	1	3	2
01	4	5	7	6
11	12	13	15	14
10	8	9	11	10

Important: the order 00, 01, 11, 10 is Gray-code order. Adjacent cells differ by only one input bit, so neighbouring 1s can be combined safely.

7.2 How the K-map is constructed

1. Write the valid counter sequence. Example: MOD-6 uses 0 → 1 → 2 → 3 → 4 → 5 → 0.
2. Convert each digit to BCD. For example, 5 = 0101 and 0 = 0000.
3. Create the present-to-next-state table. The present BCD value identifies the K-map cell.
4. Fill one map for each next-state bit. A 1 means that bit must be HIGH in the next state.
5. Mark unused states as X when permitted. These are don't-care conditions and can help form larger groups.
6. Group adjacent 1s. Groups are made in powers of two: 1, 2, 4, 8, etc.
7. Read the simplified expression. Variables that change inside a group disappear.

7.3 Why this is useful in the digital clock

The result of the K-map is a compact next-state rule. That rule is then implemented by the Arduino program. This makes the counter transitions easier to understand and helps avoid invalid values such as 60 seconds, 60 minutes or 24 hours.

7.4 Counter Types Used in the Project

Counter	Sequence	Where it is used
MOD-10	0 → 1 → … → 9 → 0	Units of seconds, minutes and normal hour units
MOD-6	0 → 1 → … → 5 → 0	Tens of seconds and tens of minutes
MOD-4	0 → 1 → 2 → 3 → 0	Hour units when hour-tens = 2
MOD-3	0 → 1 → 2 → 0	Hour-tens digit

7.5 Clear Example: MOD-6 Increment

The MOD-6 counter is a good example because only six decimal states are valid. Its BCD transition is:

0000 → 0001 → 0010 → 0011 → 0100 → 0101 → 0000

X■X■ \ X■X■	00	01	11	10
00	0	1	0	1
01	1	X	X	X
11	X	X	X	X
10	X	X	X	X

This small map shows the values for the **next least-significant BCD bit** of the MOD-6 increment operation. The X entries represent states outside the intended 0–5 sequence. When the specification allows them to be don't-cares, they can be used to make larger groups and simplify the logic.

7.6 MOD-10 Example

MOD-10 follows 0 → 1 → 2 → 3 → 4 → 5 → 6 → 7 → 8 → 9 → 0. The same K-map procedure is repeated for the four next-state bits. BCD states 10–15 are invalid decimal states and may be treated as don't-cares when the design specification permits it.

Present	Next (increment)	Present	Next (decrement)
0	1	0	9
1	2	1	0
2	3	2	1
…	…	…	…
8	9	8	7
9	0	9	8

Reading the table: the left half shows increment and the right half shows decrement. The K-map is simply the graphical method used to turn these required transitions into simpler Boolean next-state logic.

7.7 From K-Map to the Working Digital Clock

The most important point is that a K-map is not an isolated calculation. It is one stage in the complete digital-clock design. The present digit is converted to BCD, the required next state is obtained from the counter rule, the K-map simplifies the Boolean logic, and the Arduino applies that next state.

Present digit	→	**BCD present state**	→	**K-map / Boolean logic**	→	**Next BCD state**
Next BCD state	→	**7447 decoder**	→	**Seven-segment display**	→	**Visible digit**

7.8 Increment and Decrement Together

The project supports editing selected digits, so both directions are required. Increment means moving to the next valid state; decrement means moving to the previous valid state. The same K-map method is used in both cases.

Operation	Required sequence / purpose
MOD-10 increment	0–9 and return to 0
MOD-10 decrement	0 and return to 9
MOD-6 increment	0–5 and return to 0
MOD-6 decrement	0 and return to 5
MOD-4 increment/decrement	0–3 for the 20–23 hour condition
MOD-3 increment/decrement	0–2 for the hour-tens digit

7.9 Boundary Conditions and Valid Time

59 seconds → 00 seconds: seconds reset and the minute advances.
59 minutes → 00 minutes: minutes reset and the hour advances.
23:59:59 → 00:00:00: the complete clock returns to the start of the 24-hour cycle.
Hour condition: when the hour-tens digit is 2, the hour-ones digit is restricted to 0–3.

7.10 Key Points to Remember

• K-map inputs = present-state BCD bits.
• K-map outputs = required next-state bits.
• Use Gray-code order: 00, 01, 11, 10.
• Group adjacent 1s in powers of two.
• Use valid don't-care states when permitted.
• Different clock positions require different MOD counters.
• The simplified result is implemented in the Arduino program.

Summary: K-maps make the clock's state transitions easier to simplify, explain and implement. They connect the Boolean-logic theory of the project directly to the practical Arduino, BCD, 7447 and seven-segment display system.

8 Multiplexing

All six display positions share the same BCD information. Only one digit is enabled at a time. The controller writes the BCD value for digit 1, enables digit 1 briefly, disables it, then repeats the process for digits 2 through 6.

In the submitted code each digit is enabled for approximately 2 ms:

$$6 \times 2\,\text{ms} = 12\,\text{ms}$$

per complete scan, giving an approximate scan frequency of

$$f \approx \frac{1}{0.012} \approx 83.3\,\text{Hz}.$$

The human eye integrates the rapidly changing images, so the six digits appear continuously illuminated.

Multiplexing is important because otherwise every segment of every digit would need an independent control connection. The reference design describes the same principle: shared BCD inputs and one display enabled at a time.

9 Switch Debouncing

Mechanical push buttons exhibit contact bounce. When a button is pressed, the electrical signal may look like:

$$0 \to 1 \to 0 \to 1 \to 1$$

in a very short interval. If the program reacts to every transition, one physical press can be interpreted as several presses.

Common solutions include:

1. Software delay after detecting a transition.
2. Sampling the switch repeatedly and accepting a state only after it remains stable.
3. Hardware RC filtering.
4. A Schmitt-trigger input stage.

The reference project includes software debouncing as part of the intended implementation. For the present breadboard prototype, switch wiring itself was also a source of error.

10 Software Design

The submitted program has four important software blocks:

1. Time update: uses `millis()` to detect one-second intervals.
2. Cascaded counting: increments seconds, then minutes, then hours with rollover conditions.
3. BCD generation: converts a decimal digit into four output bits.
4. Display refreshing: rapidly scans the six display positions.

10.1 Why `millis()` is used

The code uses `millis()` instead of a long one-second blocking delay. This is important because the display must continue to be refreshed many times per second. A long blocking delay would interrupt multiplexing and could make the display flicker or appear dim.

11 Circuit Construction Procedure

1. Place the Arduino Uno and breadboard on a stable surface.
2. Place the six seven-segment displays and 7447 IC on the breadboard.
3. Add push buttons to the breadboard by placing both legs of the tactile switch on the same side of the breadboard gap.

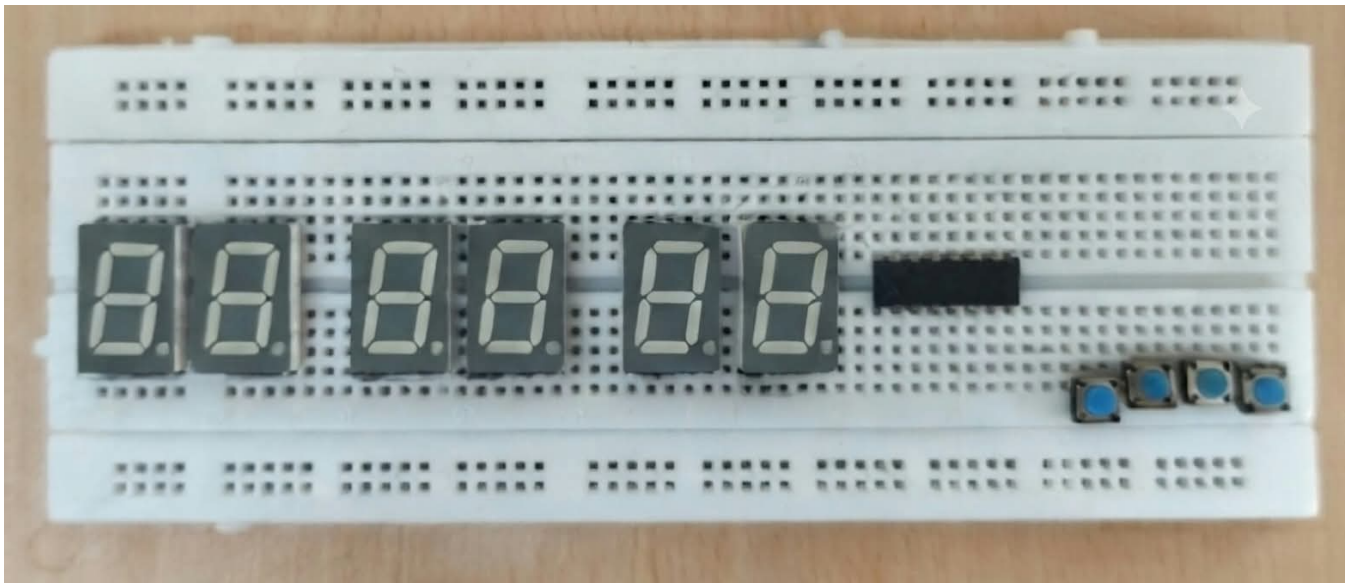


Figure 2: Initial component placement on the breadboard, shown horizontally.

4. Identify the 7447 IC orientation and pin 1 before wiring.

Circuit Construction – Step 1

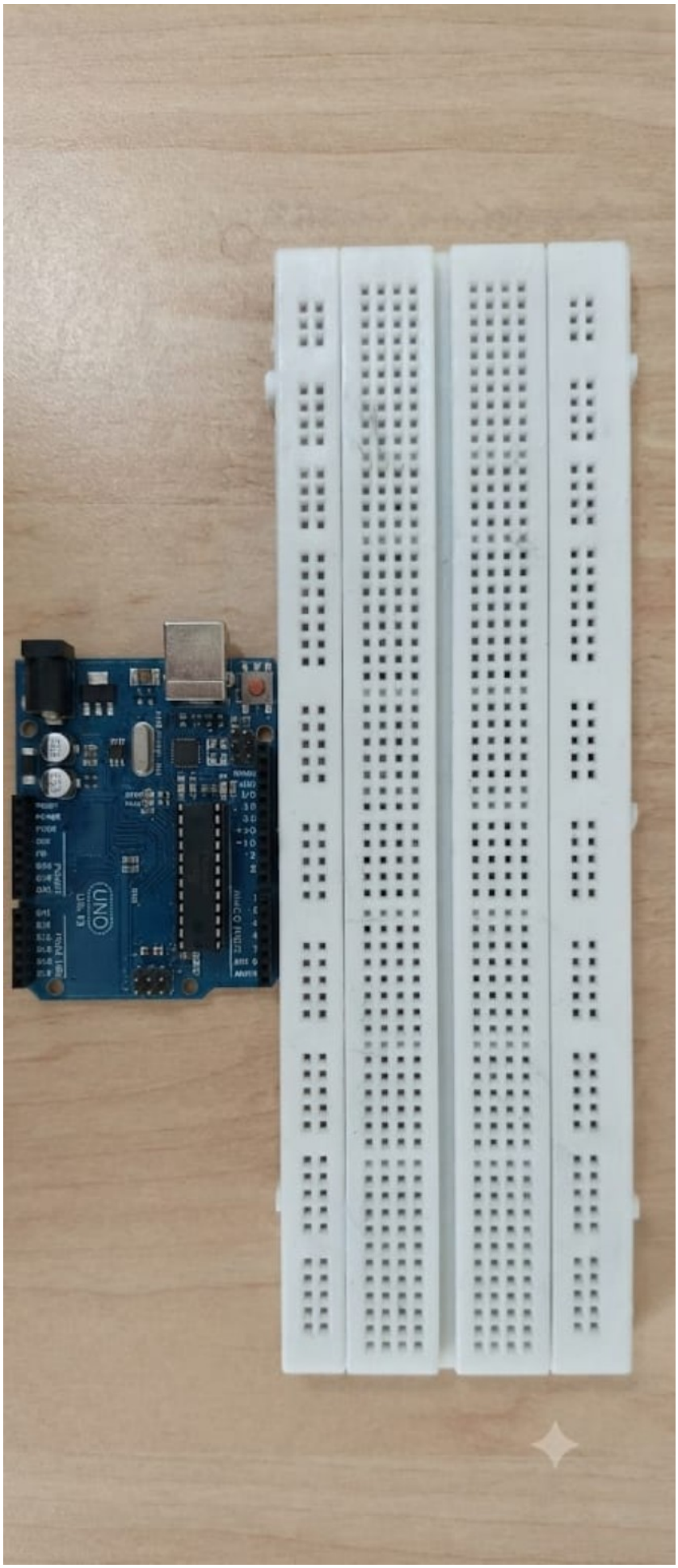


Figure: Initial A adboard setup

5. Connect all the six seven-segment displays using jumper wires such that a,b,c,d,e,f,g of D1,D2,D3,D4,D5,D6 are connected.

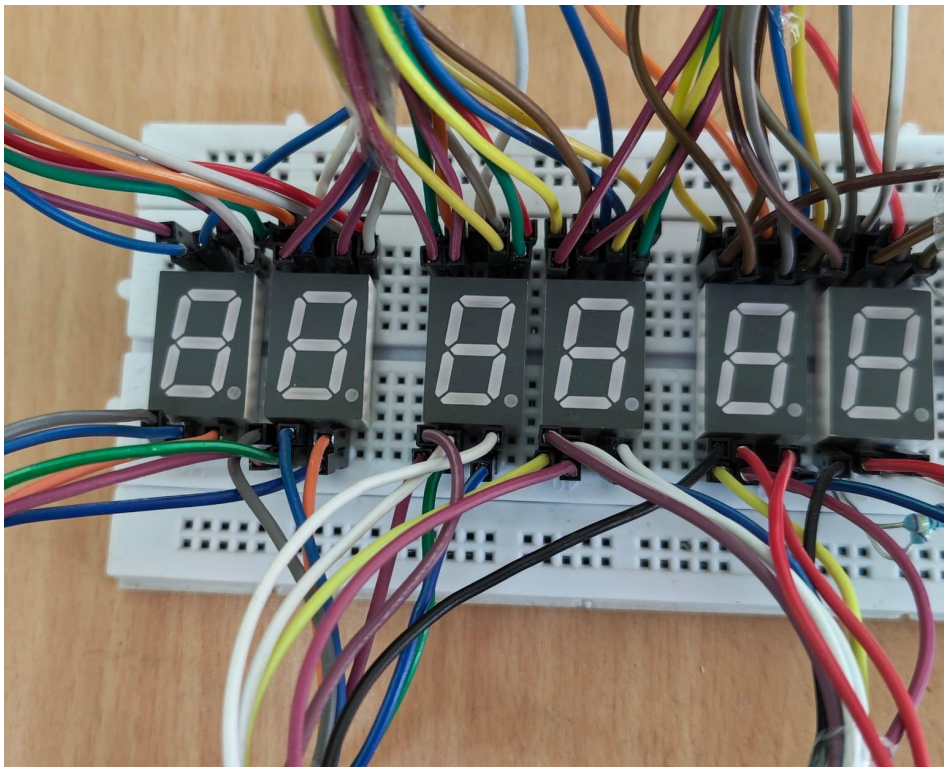


Figure 3: Wiring of the six seven-segment displays.

6. Connect Arduino D7, D4, D5 and D6 to the 7447 IC BCD inputs A, B, C and D, respectively, using jumper wires.
7. Connect the 7447 segment outputs to the seven-segment display network with appropriate current-limiting resistors.
8. Connect the six display-enable lines to Arduino D8–D13 according to the physical order required by the display mode.
9. Connect 7447 segment Vcc to Arduino 5V and GND to Arduino 0V.
10. Attach the Arduino 5V and 0V to the power rails (+ and -) on the outer edges of the breadboard.
11. Upload a simple digit-test program before running the full clock program.
12. Test digits 0–9 individually.
13. Test the six-digit multiplexing sequence.
14. Test the 00–59 second rollover.
15. Test the 00–59 minute rollover.
16. Test the 00–23 hour rollover.

17. Finally test the buttons and debouncing.
18. Add push button on the power rails.
19. Connect one leg of push button A0 of Arduino and other leg to ground.
20. This push button helps us to change the brightness as
    one tap (Lowest brightness)
    double tap (Intermediate brightness)
    triple tap (Peak brightness)

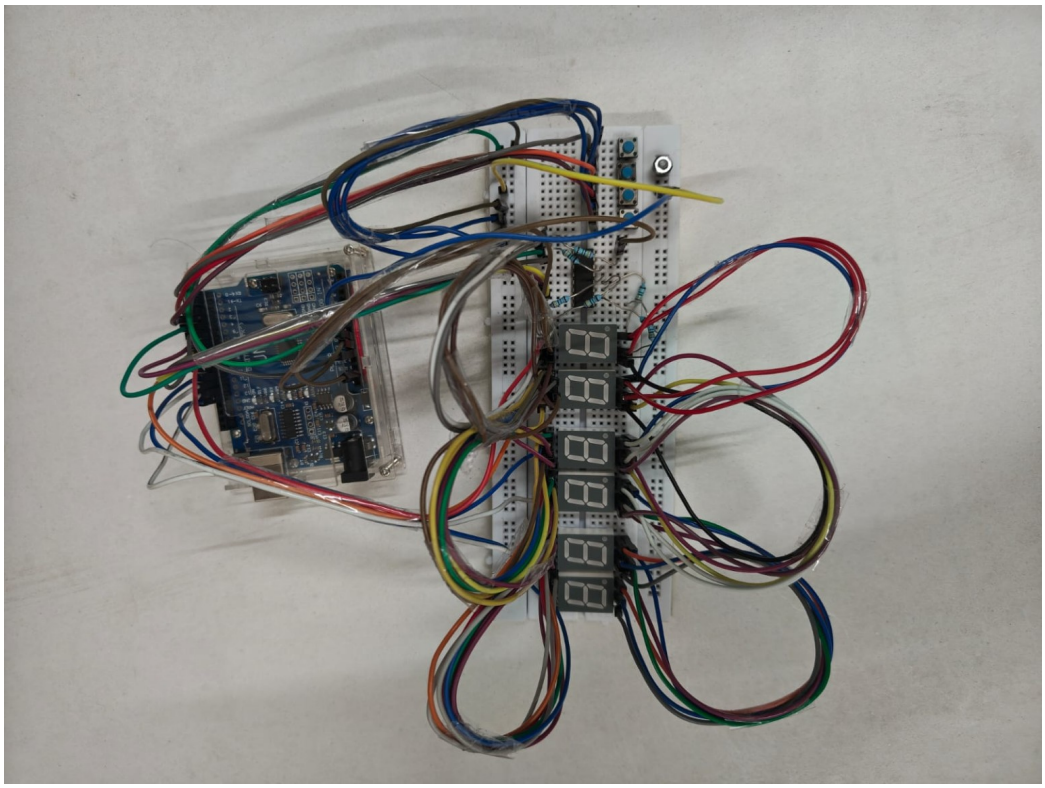


Figure 4: Complete circuit assembly with Arduino Uno connected and push-button arrangement.

12 Safety and Reliability Checks

- Verify the display type and 7447 polarity before applying power.
- Never short the 5 V supply directly to ground.
- Use current-limiting resistors appropriate to the display and driver configuration.
- Confirm IC orientation before powering the circuit.
- Disconnect USB power before major rewiring.
- Check continuity of power and ground rails before inserting the IC.

12 Errors and Corrections

Error 1: Push Button Bridge Misplacement Across Breadboard Gap

WIRING ERROR

SYMPTOM & CAUSE

Push buttons were originally placed spanning the center divider trench (gap) of the breadboard with one leg connected to the Arduino input pin and the other leg connected directly to Ground (GND) across separate isolated rail sections incorrectly, leading to floating inputs or unintended short configurations when pressed.

CORRECTION / SOLUTION

Rectification: Repositioned the push buttons so that their pins align properly on the same side/rail of the breadboard gap (or correctly bridging the trench without conflicting rail splits). Ensured correct pull-down/pull-up wiring topology where one side connects to the digital input pin (with appropriate resistor configuration) and the opposite terminal connects securely to GND.

Error 2: Breadboard Internal Pin Detachment via Resistor Over-Pressing

HARDWARE / MECHANICAL FAILURE

SYMPTOM & CAUSE

A specific resistor connecting the IC to the 7-segment display was forcefully over-pressed into the breadboard hole. This physical stress permanently deformed or detached the internal spring clip / metal contact beneath the breadboard socket. Consequently, one specific segment failed to glow across all multiplexed displays despite flawless code and external wiring.

12 Errors and Corrections (continued)

CORRECTION / SOLUTION

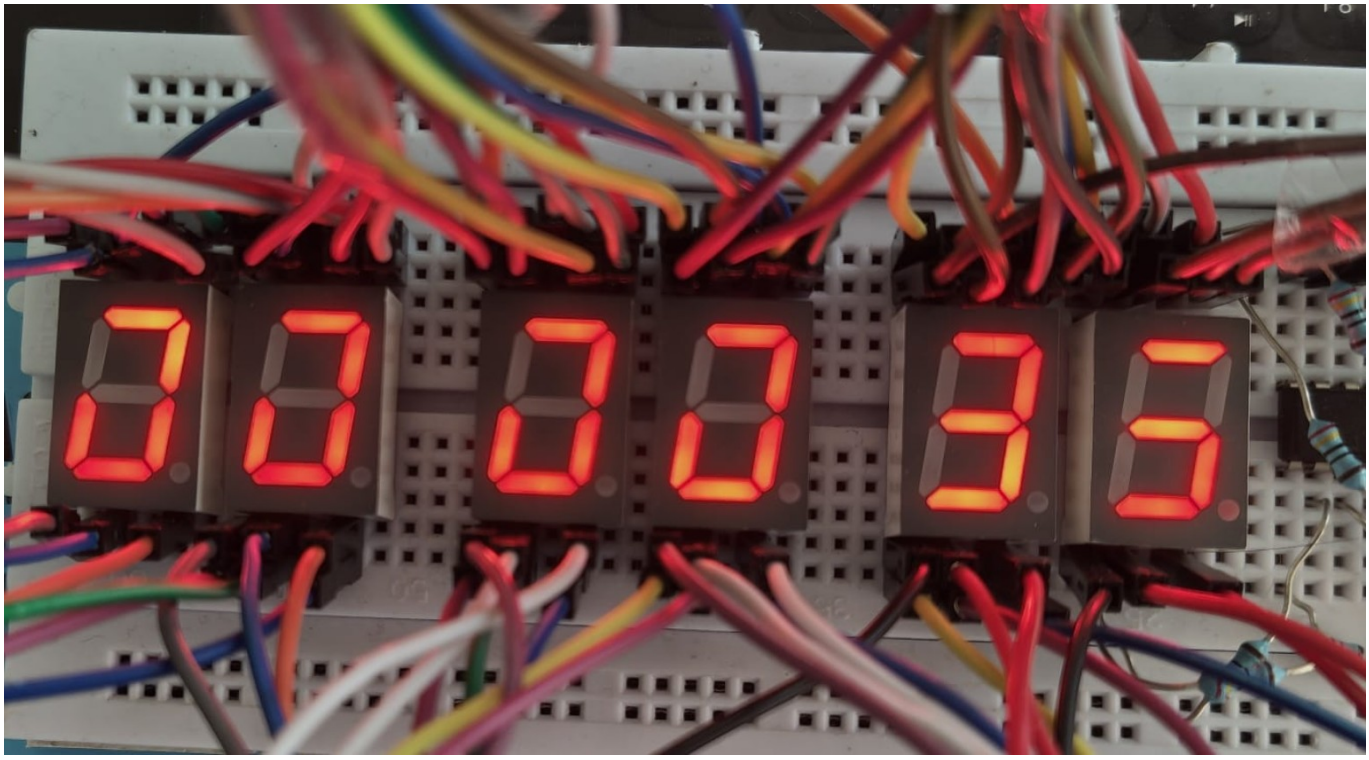
Rectification: Diagnosed the hidden hardware fault by performing continuity checks across the breadboard node. Relocated the resistor and its connected jumper wire to an adjacent, undamaged row on the breadboard, restoring stable electrical contact and full segment illumination.

Images – Errors and Corrections



Prototype/documentation image

Prototype – Digital Clock Circuit



Assembled six-digit digital clock prototype

13 Conclusion

The six-digit digital clock is a compact project that connects theoretical digital design with practical embedded-system construction. The prototype combines Arduino timing, BCD representation, 7447 decoding, seven-segment display driving and multiplexing. The clock also provides a practical context for studying state machines, K-maps and switch debouncing.

The debugging experience with the push-button wiring is an important part of the project because it demonstrates how physical circuit knowledge is required in addition to software knowledge. Brightness control through single-tap, double-tap, and triple-tap actions adds an additional user-interface feature to the clock.

14 References

1. Dhawal Saini and G. V. V. Sharma, *Digital Clock using the Arduino Framework*, Department of Electrical Engineering, Indian Institute of Technology Hyderabad, January 29, 2026.

2. Arduino documentation and Arduino Uno reference material.

3. Project source code supplied by the student for the present prototype.

A Quick Pin-and-Logic Summary

Signal	**Arduino pin**
A (1)	D7
B (2)	D4
C (4)	D5
D (8)	D6
Digit select 1	D8
Digit select 2	D9
Digit select 3	D10
Digit select 4	D11
Digit select 5	D12
Digit select 6	D13